

Laporan Hands-on 3

Konsep dasar process pada sistem operasi dibahas melalui serangkaian eksperimen Bash yang berfokus pada pembuatan process, observasi PID, monitoring status, komunikasi sederhana antarprocess, penanganan signal, dan dependency pada workflow. Rangkaian praktikum ini menempatkan Bash sebagai alat observasi langsung untuk melihat bagaimana process dibuat, dijalankan, dipantau, dan dihentikan tanpa harus menggunakan program yang kompleks.

Seluruh eksperimen memperlihatkan bahwa process management bukan sekadar menjalankan perintah lalu menunggu hasilnya. Shell juga dapat membuat background process, memeriksa apakah process tertentu masih hidup, menyinkronkan dua alur kerja sederhana, serta mengatur urutan eksekusi yang bergantung pada hasil process sebelumnya. Dengan demikian, Hands-on 3 menjadi landasan penting sebelum masuk ke topik yang lebih lanjut seperti synchronization, IPC, lifecycle management, dan automation.

Section 1 - Membuat dan Mengamati Simple Process di Bash

Bagian pertama memperkenalkan bentuk paling dasar dari process management, yaitu pembuatan sebuah process dari shell dan observasi identitasnya melalui PID. Ketika sebuah perintah dijalankan dari Bash, shell sebenarnya sedang meminta sistem operasi untuk membuat process baru. Tujuan utama section ini adalah menunjukkan bahwa process tersebut dapat dipindahkan ke background, diamati statusnya, lalu dihentikan secara manual.

Implementasi praktikum menggunakan file `task1_process_basic.sh` yang menjalankan `sleep 30` sebagai background process dengan operator `&`. PID dari process tersebut disimpan menggunakan `$!`, lalu diamati memakai `ps -f -p`. Script kemudian menunggu selama 3 detik, memeriksa kembali status process, dan akhirnya mengirim signal `kill` untuk menghentikannya. Pada hasil eksekusi yang dikerjakan untuk laporan ini, process `sleep` terlihat aktif saat observasi pertama dan kedua, lalu tidak lagi muncul setelah signal terminasi dikirim.

Dari sudut pandang sistem operasi, eksperimen ini memperlihatkan bahwa setiap process memiliki identitas yang terpisah dari shell induknya dan dapat dikendalikan melalui PID. Background execution juga menunjukkan bahwa shell tidak harus menunggu process selesai sebelum melanjutkan instruksi berikutnya. Section ini menjadi fondasi penting untuk memahami lifecycle process, observasi status, dan kontrol process secara eksplisit.

Ringkasan Eksperimen

Komponen	Informasi
File praktikum	<code>task1_process_basic.sh</code>
Process yang dijalankan	<code>sleep 30</code>
Mekanisme utama	Background process dengan <code>&</code> , observasi dengan <code>ps</code> , terminasi dengan <code>kill</code>
Identitas process	PID disimpan melalui <code>\$!</code>

Fenomena utama	Process terlihat aktif saat diamati lalu hilang setelah dihentikan
Konsep utama	Pembuatan process, PID, background execution, dan observasi status

Kesimpulan Section 1

Pembuatan simple process di Bash menunjukkan bahwa shell bukan hanya alat untuk menjalankan perintah, tetapi juga alat untuk mengontrol process secara langsung. Dengan background execution, observasi PID, dan terminasi manual, mahasiswa dapat melihat bahwa process memiliki lifecycle yang dapat diikuti dan dikelola sejak tahap paling dasar.

Section 2 - Monitoring Status Process Lain

Setelah process dapat dibuat dan diamati, langkah berikutnya adalah memeriksa apakah process lain masih berjalan atau sudah selesai. Kebutuhan seperti ini sangat umum pada automation dan workflow sederhana, karena satu task sering kali harus mengetahui status task lain sebelum mengambil keputusan. Section ini menekankan bahwa monitoring process dapat dilakukan hanya dengan memanfaatkan PID dan loop pemeriksaan sederhana.

Implementasi praktikum memakai file `task2_check_process_status.sh` yang menjalankan worker `sleep 10` sebagai background process. Script utama menyimpan PID worker, lalu melakukan polling dengan `kill -0` setiap 2 detik untuk memeriksa apakah process masih ada. Jika worker masih hidup, script mencetak status bahwa worker masih berjalan; jika tidak, loop dihentikan. Pada hasil run yang digunakan dalam laporan ini, status "Worker is still running..." muncul beberapa kali sebelum akhirnya berubah menjadi "Worker has finished." ketika process worker selesai.

Secara konseptual, section ini memperlihatkan bahwa coordination antarprocess tidak selalu memerlukan tool yang rumit. PID sudah cukup untuk menjadi acuan observasi, dan exit condition dari process dapat dijadikan dasar pengambilan keputusan. Dari sudut pandang sistem operasi, polling seperti ini memang sederhana, tetapi sangat efektif untuk memperkenalkan ide monitoring dan state checking pada process.

Ringkasan Eksperimen

Komponen	Informasi
File praktikum	<code>task2_check_process_status.sh</code>
Worker process	<code>sleep 10</code>
Teknik monitoring	<code>kill -0</code> PID di dalam loop
Interval pemeriksaan	2 detik
Fenomena utama	Script utama terus memantau worker sampai worker selesai
Konsep utama	Monitoring status process berdasarkan PID

Kesimpulan Section 2

Status sebuah process dapat dipantau dari script lain tanpa harus menghentikan process tersebut. Dengan `kill -0`, Bash dapat melakukan existence check secara ringan dan menggunakan hasilnya

untuk mengatur alur program. Section ini memperlihatkan dasar monitoring process yang nanti sangat berguna pada service supervision dan workflow automation.

Section 3 - Interaksi Antarprocess dengan Signal File

Komunikasi antarprocess pada tahap awal sering kali lebih mudah dipahami melalui media yang konkret, salah satunya file di filesystem. Alih-alih menggunakan shared memory, pipe, atau socket, dua process pada section ini berkoordinasi menggunakan sebuah signal file sederhana. Tujuan utamanya adalah menunjukkan bahwa interaksi antarprocess dapat dibangun melalui perubahan keadaan yang dapat diamati bersama.

Implementasi menggunakan file `task3_process_interaction.sh` yang menjalankan dua alur, yaitu producer dan consumer. Producer bekerja selama 5 detik lalu membuat file `/tmp/process_done.signal` yang berisi tanda bahwa pekerjaan telah selesai. Consumer berjalan bersamaan dan terus memeriksa keberadaan file tersebut setiap 1 detik. Pada run yang dikerjakan untuk laporan ini, consumer beberapa kali mencetak bahwa signal belum ditemukan, lalu melanjutkan ketika file muncul dan akhirnya menampilkan isi file yang ditulis producer.

Eksperimen ini penting dari sudut pandang sistem operasi karena menunjukkan bahwa synchronization sederhana dapat dibangun dari shared external state, bukan hanya dari memory di dalam satu process. Filesystem di sini berperan sebagai media komunikasi tidak langsung. Walaupun metode ini bukan IPC paling efisien, section ini sangat efektif untuk memperlihatkan konsep wait-until-condition dan coordination antarprocess secara mudah diamati.

Ringkasan Eksperimen

Komponen	Informasi
File praktikum	<code>task3_process_interaction.sh</code>
Process yang terlibat	producer dan consumer
Media komunikasi	File <code>/tmp/process_done.signal</code>
Mekanisme utama	Producer membuat file, consumer melakukan polling file
Fenomena utama	Consumer menunggu sampai signal file muncul lalu membaca isinya
Konsep utama	Inter-process communication sederhana berbasis filesystem

Kesimpulan Section 3

Dua process dapat berinteraksi tanpa harus berbagi memory secara langsung. Dengan menggunakan signal file, producer dan consumer dapat disinkronkan berdasarkan kondisi yang terlihat bersama. Section ini menegaskan bahwa komunikasi antarprocess dapat dibangun dari mekanisme yang sangat sederhana selama ada medium bersama yang dapat diobservasi.

Section 4 - Mengamati Process Lifecycle

Sebuah process tidak hanya memiliki titik mulai dan titik akhir, tetapi juga lifecycle yang dapat diamati dan dikendalikan. Section ini menyoroti bahwa process yang sedang berjalan dapat menerima signal,

menjalankan cleanup, lalu keluar secara terkontrol. Pemahaman seperti ini penting karena banyak service nyata tidak boleh dihentikan secara sembarangan tanpa memberi kesempatan untuk membersihkan resource yang sedang dipakai.

Implementasi praktikum menggunakan file `task4_process_lifecycle.sh` yang memasang trap untuk SIGTERM dan SIGINT, lalu menjalankan loop tak berhingga yang mencetak status iterasi setiap 2 detik. Ketika process ini dijalankan, PID-nya dapat diamati dari terminal lain memakai `ps -f -p`. Pada eksekusi yang dipakai untuk laporan ini, process berjalan beberapa iterasi, kemudian menerima SIGTERM, mencetak pesan bahwa signal diterima, melakukan cleanup, dan keluar dengan rapi.

Dari perspektif sistem operasi, section ini memperjelas bahwa signal handling adalah bagian penting dari lifecycle management. Process tidak selalu berakhir karena selesai secara normal; ia juga bisa berakhir karena interupsi dari luar. Dengan trap, Bash dapat mengubah penghentian yang semula abrupt menjadi controlled termination. Hal ini sangat relevan untuk memahami service shutdown, cleanup handler, dan kontrol lifecycle yang lebih matang.

Ringkasan Eksperimen

Komponen	Informasi
File praktikum	<code>task4_process_lifecycle.sh</code>
Mekanisme utama	Loop aktif dengan trap untuk SIGTERM dan SIGINT
Observasi process	PID process diamati dari shell lain menggunakan <code>ps</code>
Interval aktivitas	2 detik per iterasi
Fenomena utama	Process menerima signal, menjalankan cleanup, lalu exit
Konsep utama	Lifecycle process, signal handling, dan controlled termination

Kesimpulan Section 4

Lifecycle process dapat diamati lebih jelas ketika process menerima signal dari luar dan meresponsnya secara terkontrol. Dengan memasang handler melalui trap, process tidak hanya berhenti, tetapi juga dapat menyelesaikan langkah cleanup terlebih dahulu. Section ini memperkuat hubungan antara process control dan pengelolaan resource yang aman.

Section 5 - Dependency Antarprocess pada Workflow Sederhana

Bagian penutup Hands-on 3 menekankan bahwa satu process sering kali bergantung pada hasil process lain sebelum dapat melanjutkan pekerjaannya. Pola seperti ini umum pada workflow data, automation script, dan pipeline sederhana. Ide utamanya adalah bahwa correctness tidak hanya ditentukan oleh isi tiap langkah, tetapi juga oleh urutan eksekusinya.

Implementasi memakai file `task5_process_dependency.sh` dengan tiga tahap utama: `download_data`, `validate_data`, dan `generate_report`. Tahap pertama dijalankan sebagai background process, lalu script utama menunggu dengan `wait` sampai data selesai dibuat. Setelah itu validasi memeriksa apakah file tersedia dan tidak kosong. Jika validasi berhasil, report dibuat dengan

perintah penghitung jumlah baris pada file data. Pada run yang digunakan untuk laporan ini, workflow berjalan sukses dan report akhir menunjukkan bahwa file data berisi 3 baris.

Section ini memperlihatkan bentuk paling sederhana dari dependency control di sistem operasi dan shell scripting. `wait` berfungsi sebagai mekanisme sinkronisasi urutan, sedangkan exit status dari validasi menjadi dasar untuk memutuskan apakah workflow boleh dilanjutkan. Dengan demikian, eksperimen ini menegaskan bahwa Bash tidak hanya cocok untuk command kecil, tetapi juga mampu membangun workflow yang aman dan terstruktur.

Ringkasan Eksperimen

Komponen	Informasi
File praktikum	<code>task5_process_dependency.sh</code>
Tahap workflow	<code>download_data</code> , <code>validate_data</code> , <code>generate_report</code>
Mekanisme sinkronisasi	<code>wait</code> terhadap process pengumpulan data
Shared artifact	<code>/tmp/sample_data.txt</code> dan <code>/tmp/report.txt</code>
Hasil akhir	Report menunjukkan 3 baris data pada file sumber
Konsep utama	Dependency antarprocess, validasi hasil, dan sequential workflow

Kesimpulan Section 5

Dependency antarprocess membuat urutan eksekusi menjadi bagian penting dari correctness workflow. Dengan `wait` dan pemeriksaan hasil validasi, Bash dapat memastikan bahwa setiap langkah hanya berjalan ketika prasyaratnya sudah terpenuhi. Section ini menutup Hands-on 3 dengan menunjukkan bahwa process control sangat erat kaitannya dengan automation yang aman dan dapat diprediksi.

Penutup

Berdasarkan seluruh rangkaian praktikum ini, dapat dilihat bahwa pembelajaran process pada Bash berkembang dari konsep paling dasar hingga bentuk coordination yang lebih terstruktur. Praktikum dimulai dari pembuatan background process dan observasi PID, berlanjut ke monitoring status, komunikasi antarprocess berbasis file, pengamatan lifecycle melalui signal, hingga dependency pada workflow sederhana. Seluruh bagian ini menunjukkan bahwa bahkan shell script yang relatif singkat sudah cukup untuk memperlihatkan konsep penting dalam sistem operasi secara langsung.

Secara keseluruhan, Hands-on 3 menegaskan bahwa process management bukan hanya soal menjalankan program, tetapi juga soal observasi, kontrol, sinkronisasi, dan urutan kerja yang benar. Pemahaman ini menjadi fondasi yang kuat untuk topik-topik berikutnya seperti concurrency, synchronization, shared resource, deadlock, dan automation yang lebih kompleks.